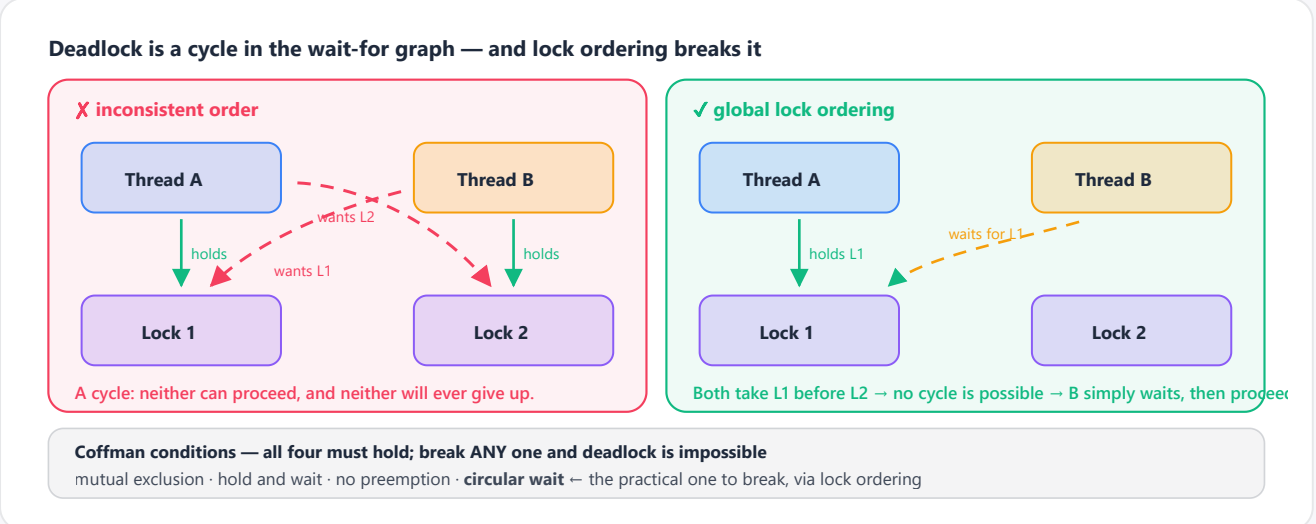
Deadlock, Livelock and Starvation

Why it matters

A deadlock is the most dramatic concurrency failure: the application does not crash, does not log anything, and simply stops. There is no stack trace to search for, because nothing threw. Interviewers ask because the fix is a design discipline rather than a code change, and because diagnosing one requires knowing how to read a thread dump — a genuinely practical skill.

The concept, simply

Deadlock is two or more threads each holding a lock the other needs, so none can proceed. It requires four conditions to hold simultaneously — the *Coffman conditions* — and breaking any one makes deadlock impossible:



The cycle is the whole problem. Enforce one global order in which locks are acquired and a cycle becomes structurally impossible.

- **Mutual exclusion** — a resource is held exclusively.

- **Circular wait** — a cycle in the wait-for graph. *This is the one you break in practice.*

How it works

Two related failures share the "nothing is progressing" symptom and are worth distinguishing:

- **Livelock** — threads are running, responding to each other, and making no progress. Two people stepping aside in a corridor, repeatedly. Common with naive retry-on-conflict logic where everyone backs off in lockstep.
- **Starvation** — a thread never gets the resource because others keep taking it. Caused by unfair locks under heavy load, or by low-priority threads that never get scheduled.

The **primary prevention technique** is a global lock ordering: define a total order over your locks and always acquire them in that order. If every thread takes Lock1 before Lock2, no cycle can form. When the objects have no natural order, use their identity hash codes as a tiebreak.

Code examples

The classic bank transfer deadlock:

JAVA Copy

```java
// ✗ Deadlocks when two transfers run in opposite directions simultaneousl
//    transfer(A, B) locks A then wants B
//    transfer(B, A) locks B then wants A
public void transfer(Account from, Account to, BigDecimal amount) {
    synchronized (from) {
        synchronized (to) {
            from.debit(amount);
            to.credit(amount);
        }
    }
}

// ✓ Impose a consistent order using a unique, stable id.
```

```
    synchronized (first) {
        synchronized (second) {
            from.debit(amount);
            to.credit(amount);
        }
    }
}
// Now every thread locks the lower id first, so no cycle can form.
// ⚠ Guard against from == to, which would otherwise self-deadlock on a
//   non-reentrant lock (intrinsic locks are reentrant, so this one surviv
```

The timeout approach, when ordering is impractical:

JAVA Copy

```
// tryLock with a timeout breaks "hold and wait" — a thread that cannot ge
// everything gives up what it has and retries, so no cycle persists.
public boolean transfer(Account from, Account to, BigDecimal amount)
        throws InterruptedException {
    long deadline = System.nanoTime() + TimeUnit.SECONDS.toNanos(1);
    while (System.nanoTime() < deadline) {
        if (from.lock().tryLock(50, TimeUnit.MILLISECONDS)) {
            try {
                if (to.lock().tryLock(50, TimeUnit.MILLISECONDS)) {
                    try {
                        from.debit(amount);
                        to.credit(amount);
                        return true;
                    } finally { to.lock().unlock(); }
                }
            } finally { from.lock().unlock(); }   // ✓ release before retr
        }
        // ⚠ Randomised backoff — without it, two threads can retry in
        //   lockstep forever, which is LIVELOCK rather than deadlock.
        Thread.sleep(ThreadLocalRandom.current().nextInt(10, 50));
    }
```

Diagnosing one — this is what the interviewer wants to hear:

BASH Copy

```bash
jcmd <pid> Thread.print     # the JVM DETECTS monitor deadlocks and names t
```

TEXT Copy

```text
Found one Java-level deadlock:
=============================
"thread-1":
  waiting to lock monitor 0x00007f8 (object 0x76b2f1234, a com.app.Account),
  which is held by "thread-2"
"thread-2":
  waiting to lock monitor 0x00007f9 (object 0x76b2f9999, a com.app.Account),
  which is held by "thread-1"

// The JVM finds cycles of INTRINSIC locks automatically. It does NOT detect
// deadlocks involving ReentrantLock in the same way — for those, read the
// stack traces for threads parked in AQS. ThreadMXBean.findDeadlockedThreads
// covers both and can be wired into a health check.
```

The subtler deadlock, which is far more common in real systems:

JAVA Copy

```java
// ✗ Calling unknown code while holding a lock. The listener might acquire
//   another lock — or call back into this class — and you have no idea.
synchronized (this) {
    for (Listener l : listeners) l.onChange(value);   // ALIEN METHOD
}
```

```
List<Listener> snapshot;
synchronized (this) {
    snapshot = new ArrayList<>(listeners);
}
for (Listener l : snapshot) l.onChange(value);        // no lock held
```

Under the hood

The JVM detects but never resolves. Thread dumps identify cycles of intrinsic monitors, and `ThreadMXBean.findDeadlockedThreads()` covers `Lock`-based ones too — but nothing breaks the cycle. Java has no lock preemption and no transaction rollback, so a deadlocked application stays deadlocked until it is restarted. Prevention is the only real strategy.

Database deadlocks are different, and the contrast is worth drawing because it comes up in Spring interviews (Chapter 12). A database *detects* deadlock and kills one transaction as the victim, which then rolls back and can be retried. That is why `DeadlockLoserDataAccessException` is retryable while a JVM deadlock is fatal. The same lock-ordering discipline prevents both — updating rows in a consistent order is the database equivalent.

Deadlocks are almost impossible to test for, because they require a specific interleaving that may occur once in millions of runs. That is precisely why the discipline matters: consistent lock ordering, never calling alien code under a lock, keeping critical sections small, and preferring higher-level constructs — a `BlockingQueue` or a `ConcurrentHashMap` — over hand-written multi-lock protocols. Most deadlocks are prevented by not writing the lock at all.

Common mistakes

- **Acquiring locks in different orders in different methods.** The direct cause of nearly every deadlock.
- **Calling alien code while holding a lock.** Listeners and callbacks can acquire anything.
- **Holding a lock across I/O.** Turns contention into an outage.

- **Retrying without randomised backoff.** Converts deadlock into livelock.
- **Assuming testing will find it.** Deadlocks need a rare interleaving.
- **Catching and continuing on `tryLock` failure without releasing.** You keep what you hold and nothing improves.
- **Nested synchronised blocks by reflex.** Every added lock multiplies the ordering constraints.

Interview lens

> **Q: "What is a deadlock and how do you prevent it?"**
> A: Two or more threads each holding a lock the other needs, so none can proceed. It needs all four Coffman conditions — mutual exclusion, hold and wait, no preemption, circular wait — and breaking any one prevents it. In practice I break circular wait with a global lock ordering: define a total order, usually by a stable id, and always acquire in that order. Where ordering is impractical, `tryLock` with a timeout and randomised backoff breaks hold-and-wait instead.

> **Q: "How do you diagnose one in production?"**
> A: Take a thread dump with `jcmd <pid> Thread.print` or `jstack`. The JVM detects cycles of intrinsic monitors and prints a "Found one Java-level deadlock" section naming the threads and the objects. For `ReentrantLock`-based deadlocks it does not report that way, so I read the stacks for threads parked in AQS, or call `ThreadMXBean.findDeadlockedThreads()`, which covers both and can be exposed as a health check.

> **Q: "Deadlock vs livelock vs starvation?"**
> A: Deadlock — threads are blocked and nothing runs. Livelock — threads are running and responding to each other but making no progress, typically retry logic backing off in lockstep. Starvation — a thread runs but never gets the resource because others keep winning it, usually from unfair locks under load. Deadlock is

the only one where the threads are genuinely stopped, so it is the one a thread dump identifies immediately.

Q: "Can the JVM recover from a deadlock?"

A: No. It can detect and report one, but there is no lock preemption and no rollback, so the threads stay stuck until the process is restarted. That is the contrast with a database, which detects deadlock, picks a victim transaction and rolls it back — which is why a database deadlock is retryable and a JVM deadlock is fatal.

Q: "Two threads transfer money between the same two accounts in opposite directions. What happens?"

A: Classic deadlock — each locks its source and waits for the other's. The fix is to order the locks by something stable and unique, such as account id, so both threads always lock the lower id first and no cycle can form. I would also guard the same-account case, and in a real system prefer a single database transaction with consistent row ordering over application-level locks.

Q: "Why is calling a listener while holding a lock dangerous?"

A: Because it is code you do not control. The listener may acquire another lock, call back into your class, or block on I/O — all while your lock is held, and you cannot reason about the ordering it introduces. The fix is to copy the listener list inside the lock and invoke them outside it, so no alien code ever runs under your lock.

Cheat-card

- Deadlock = **cycle in the wait-for graph**. Four Coffman conditions; break any one.
- Primary fix: **global lock ordering** by a stable id.
- Alternative: `tryLock` + timeout + **randomised backoff** (without it → livelock).
- **Livelock** = running, no progress. **Starvation** = never gets the resource.

- **The JVM never recovers** — no preemption, no rollback. Databases do (retryable).
- **Never call alien code under a lock** — copy the list, invoke outside.
- Never hold a lock across I/O. Prefer `BlockingQueue` / `ConcurrentHashMap` over hand-rolled multi-lock protocols.

QUICK CHECK — no marks, just for you.

1. Which Coffman condition is the one you break in practice to prevent deadlock?

- ◯ Mutual exclusion
- ◯ No preemption
- ◯ Circular wait — enforce a global lock ordering so a cycle cannot form
- ◯ Hold and wait, by never holding more than one lock

Check answer

2. Can the JVM recover from a deadlock?

- ◯ Yes — it kills one thread as a victim, like a database
- ◯ Yes, after a timeout
- ◯ Only if the locks are ReentrantLocks
- ◯ No — it can detect and report one in a thread dump, but there is no lock preemption and no rollback, so the threads stay stuck until the process restarts

Check answer

3. Why is a callback while holding a lock dangerous?

- It is code you do not control — it may acquire another lock, call back into your class, or block on I/O, all while your lock is held
- Because listeners always run on another thread
- Because callbacks cannot be synchronized
- It is not dangerous if the listener is final

Check answer

Previous
Locks: ReentrantLock, ReadWriteLock and Conditions

Next
Executors and Thread Pools

LEARN MORE

ARTICLES & DOCS

Deadlock, starvation and livelock — The Java Tutorials
docs.oracle.com

ThreadMXBean.findDeadlockedThreads — API docs
docs.oracle.com

VIDEOS



Java Multithreading: Synchronization, Locks, Executors, Deadlock, CountdownLatch & CompletableFuture
Engineering Digest

Hand-picked references for this lesson.